

The Tolkien RAG Chatbot — A Complete Guide

This guide explains, in simple words, exactly how the **Lorekeeper of the Shire** project works. It walks through the whole system step by step (from the moment your PDF books are loaded, all the way to the final answer you see on screen), explains *why* each piece exists, and then finishes with ideas for making the whole project even better.

Every technical idea is explained twice: first in plain English, and then tied directly to this project with realistic examples from **The Hobbit**.

1. The Big Picture: What Does This Program Do?

This is a **RAG** chatbot. RAG stands for *Retrieval-Augmented Generation*. It is a fancy way of saying:

Instead of asking a robot to answer from everything it "remembers" (which it often forgets or makes up), we first **search a real library** of documents for the most relevant paragraphs, and then let the AI **write its answer using only those paragraphs**.

Think of it like asking a librarian for help:

1. **You ask a question** ("Who is Beorn?").
2. **The librarian runs to the shelves** and finds the most relevant pages about Beorn (*retrieval*).
3. **The librarian reads those pages** and writes you an answer based only on them (*generation*).

This project has three main parts:

Part	File	Plain-English job
The bookshelf	<code>data/*.pdf</code>	The source books (hundreds of pages of Tolkien)
The library index	<code>vector_db.py</code> , <code>chroma_db/</code>	Reading the PDFs, organizing the content, and searching it quickly
The librarian (writer)	<code>main.py</code>	Taking the found passages and writing the final answer with an AI model

Everything runs **locally** on your computer. The heavy AI models (Qwen2.5 for writing, `mxlbai-embed-large` for searching) run through a tool called **Ollama**, and one optional step (Cohere reranking) uses an online API only if you provide a key.

The full pipeline in one picture

```
data/*.pdf  (the books)
  |
  v
[1] pymupdf reads every page's text
  |
```

```

      v
[2] smart_split splits text into chapters / sections
      |
      v
[3] RecursiveCharacterTextSplitter cuts long text
    into small ~800-character chunks
      |
      v
[4] Embeddings turn each chunk into numbers (vectors)
    --> stored permanently in chroma_db/
    (BM25 keyword index built from the same chunks)
      |
      v
      A question from the user
      |
      v
[5] Hybrid search = semantic (Chroma) 60% + keyword (BM25) 40%
      |
      v
[6] Optional: Cohere reranking re-orders + drops irrelevant
      |
      v
[7] format_passages() builds a "context" text with book/chapter headers
      |
      v
[8] Qwen2.5 (Ollama) writes the final answer using only that context
      |
      v
    print(answer)

```

The rest of this guide explains each numbered step. **Note:** steps 1-4 happen only the first time you run the program (when they build the library). Steps 5-8 happen on *every* question you ask.

2. The Workflow of the Whole System

This section answers the question: **what happens after the data is loaded?** It follows the exact order the program runs in, starting from the raw PDF files.

2.1 Step 0 — Where does the data live?

The source material sits in the `data/` folder as PDF files (in this build, nine PDFs covering The Hobbit and related documents). The code expects them there:

```

pdf_files = glob.glob("data/*.pdf")          # find all PDFs
if not pdf_files:
    raise FileNotFoundError("No PDF files found in data/...")

```

Plain English: the program looks inside `data/` for every `.pdf` file. If the folder is empty, it politely refuses to start instead of crashing in a confusing way later.

Hobbit example: if `data/` contains `the_hobbit.pdf`, the program will find it here. If you empty the folder on purpose, this check stops the program early with a clear message: *"put your Tolkien PDF files in the data/ folder first."*

2.2 Step 1 — Reading the PDFs (pymupdf)

A PDF is a container for pages; the words are not always easy to grab. The program opens each PDF and reads every page's text:

```
with pymupdf.open(pdf) as doc:
    for page in doc:
        page_text = page.get_text() # extract the words from this page
        if page_text.strip():
            text += "\n" + page_text.strip()
```

Plain English: `pymupdf` unlocks the PDF and pulls every page's text out into one long string. Pages that are blank are skipped. This turns a binary PDF file into a plain-text version the rest of the program can work with.

Hobbit example: `the_hobbit.pdf` becomes one giant text that starts with *"In a hole in the ground there lived a hobbit..."* and flows all the way to *"...and he went on, to find a good and prosperous day."*

2.3 Step 2 — Smart Splitting into Chapters (`smart_split`)

A whole book is far too much text to handle as one unit. The first thing the program does is break it into **logical blocks** — usually chapters. The function `smart_split()` decides *how* to split based on what the text looks like:

```
def smart_split(text, source, book_name):
    if book_name == "The Hobbit" and re.search(CHAPTER_RE, text):
        return split_by_chapters(text, source, book_name) # has "Chapter N"
    if "===" in text:
        return split_by_sections(text, source, book_name) # has === markers
    return [Document(page_content=text.strip(), ...)] # otherwise: whole text
```

It checks, in order:

1. **Is it The Hobbit and does it contain "Chapter N"?** Then split on `Chapter\s+\d+` (a regex that matches things like `Chapter 5`). The `split_by_chapters` function uses a regex with a *lookahead* so each chunk *starts* right before a chapter heading:

```
chapters = re.split(r"(?=Chapter\s+\d+)", text, flags=re.IGNORECASE)
```

2. **Does the text contain `===` markers?** Some documents (like the supplementary Tolkien notes) use `===` as a separator. Then split on those instead, and use the first line of each section as its name:

```
first_line = section.split("\n", 1)[0].strip()
```

3. **Neither?** Then treat the entire document as a single block.

Each block becomes a LangChain `Document` — a small package that holds the **text** (`page_content`) plus **metadata** (where it came from):

```
Document(page_content=block, metadata={
    "source": source, # the PDF filename
    "book": book_name, # e.g. "The Hobbit"
    "chapter": chapter_label, # e.g. "Chapter 5: Riddles in the Dark"
```

```
}}
```

Why metadata matters: it is the memory of *where* every passage came from. Later, the answer will be labeled like `[The Hobbit-Chapter 5: Riddles in the Dark]` — that header is printed straight from this metadata.

Hobbit example: The Hobbit text triggers rule 1, because every chapter is headed `Chapter 1: An Unexpected Party`, `Chapter 2: Roast Mutton`, and so on. The program returns 19 separate chapters (plus any preface text before the first heading). A supplementary file with `===` separators triggers rule 2 instead and is split into named sections.

2.4 Step 3 — Chunking with `RecursiveCharacterTextSplitter`

Even a single chapter is usually too long. Why does size matter? Because of how searching works: each chunk becomes one "index card" in the library. If a card is an entire chapter, it may talk about Gollum, the ring, riddles, and escape all at once — its "meaning" gets blurry, and a search for *"riddles"* returns a card that is mostly about something else.

So the program cuts each chapter into **chunks** of about 800 characters, with 200 characters of overlap so nothing important gets sliced in half:

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=200,
    separators=["\n\n", "\n", ". ", " ", ""],
)
```

This splitter is called **recursive** because it tries the biggest separator first, then smaller and smaller ones, only breaking into smaller pieces if it has to:

1. Prefer splitting between **paragraphs** (`\n\n`).
2. If a paragraph is still too big, split at **line breaks** (`\n`).
3. Still too big? Split at **sentence endings** (`.`).
4. Then **words** ().
5. As a last resort, split mid-word at **characters**.

This way the breaks land at the most natural points — between sentences and paragraphs, not in the middle of a word.

The 200-character **overlap** means each chunk shares a little of its neighbors' tail/head. If a sentence happens to straddle a cut, the next chunk still contains the full sentence, so no information is lost at the seam.

Hobbit example: Chapter 5, *Riddles in the Dark* (~10,000 words in the book), is cut into ~15-20 chunks of about 800 characters. A chunk near the beginning might contain the riddle *"What has roots as nobody sees, is taller than trees, up, up it goes, and yet never grows?"* and the next chunk begins with Gollum's hissed *"Full and clever... he is."* Thanks to the overlap, Gollum's reply is never cut off mid-sentence.

2.5 Step 4 — Embeddings and the Vector Store

Now comes the part that makes "semantic" search possible. The program converts each chunk into a list of numbers called an **embedding**:

```
embeddings = OllamaEmbeddings(model="mxbai-embed-large", ...)
```

Plain English — what is an embedding? Imagine you translate each passage into a "meaning fingerprint." The fingerprint is a list of ~1000 numbers. Passages with similar *meaning* get similar fingerprints, even if they use completely different words. The AI model that makes these fingerprints is `mxbai-embed-large`, running locally through Ollama.

The fingerprints are stored in a **vector database** called **Chroma** (persisted on disk in `chroma_db/`):

```
vector_store = Chroma(
    collection_name="tolkien_lore",
    persist_directory=db_location,
    embedding_function=embeddings,
)
```

When the database is first built, the program:

1. Generates a random `uuid` for every chunk (its library-card number).
2. Embeds chunks in **batches of 50** so the computer's memory doesn't overflow, printing progress as it goes:

```
for i in range(0, len(chunk_texts), batch_size):
    batch = chunk_texts[i : i + batch_size]
    vectors.extend(embeddings.embed_documents(batch))
    print(f"  embedded {done}/{len(chunk_texts)} chunks", flush=True)
```

3. Writes everything into Chroma.

Why embed at all? Because searching by exact words misses meaning. A question like *"Why was Bilbo surprisingly bold during the riddles?"* never contains the strings *"full and clever"* or *"incredibly brave"*, but a good embedding knows those phrases are about the same idea.

Hobbit example: the question *"What did the dwarves eat at Beorn's house?"* will embed *near* the passage *"They ate twice a day where they could get it — and they got it now, for honey and cream and bread and butter"* even though the exact words "eat" and "Beorn's house" aren't in it. That is semantic similarity: the vectors are close.

How the search picks a chunk — Chroma `as_retriever`: to answer a question, the same embedding model converts the *question* into a fingerprint, then Chroma measures how *close* the question's fingerprint is to every chunk's fingerprint and returns the `k` closest ones:

```
chroma_retriever = vector_store.as_retriever(search_kwargs={"k": k})
```

Hobbit example: asking *"Who is a skin-changer that can become a bear?"* returns the Beorn passages first, because Beorn IS described exactly that way in chapter 7, *Queer Lodgings*.

2.6 Step 5 — What is BM25, and why does the project also use it?

First, what is BM25? BM25 (Best Matching 25) is a classic **keyword search** algorithm, the same family of maths used by older search engines. Instead of thinking about *meaning*, it counts **words**. For a given question, BM25 scores every document by:

- how many times each *search word* appears in it (more = better),
- giving rare, specific words more importance than common filler words,
- and slightly *penalizing* documents where a word keeps repeating, so a page that mentions "the" 500 times isn't favored.

The result is a score; higher score = better keyword match. The project uses `rank_bm25`:

```
from rank_bm25 import BM25Okapi

tokenized_docs = [doc.page_content.lower().split() for doc in documents]
bm25 = BM25Okapi(tokenized_docs)
```

The text is lowered and split on spaces ("tokenized") before indexing. NLTK downloads its `punkt` model quietly to help with future tokenization work, and the resulting index is wrapped in a `BM25Retriever`:

```
class BM25Retriever(BaseRetriever):
    def _get_relevant_documents(self, query):
        tokenized_query = query.lower().split()
        scores = self.bm25.get_scores(tokenized_query)
        top_k_idx = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)
        [:self.k]
        return [self.documents[i] for i in top_k_idx]
```

It scores every document, sorts by score descending, and returns the top `k` (default 6).

Where and when is BM25 used in this project? BM25 is the *second* search engine in the pipeline. Semantic search (Chroma) is excellent at meaning, but it is famously bad at **exact names and rare spellings** — a question containing `Finwë` or `Gríma` might embed near a wrong passage. Keyword search nails exact strings. So the project runs **both** and merges the results.

Hobbit example: ask "Who shot Smaug with the Black Arrow?" BM25 counts exact hits — `Smaug`, `Black Arrow`, `Bard` all appear literally in chapter 14, *Fire and Water*. Even if the embedding got a little fuzzy, BM25 will pull that exact chapter to the top. Conversely, ask "What happened to the treasure after the dragon died?" — no chunk literally says "the dragon died" + "the treasure" together, so semantic search does the heavy lifting and BM25 adds a supporting vote.

2.7 Step 6 — Merging both: the `HybridRetriever`

The `HybridRetriever` runs semantic and keyword search side by side and **fuses** the two lists:

```
class HybridRetriever(BaseRetriever):
    chroma_weight: float = 0.6 # semantic gets 60% of the vote
    bm25_weight: float = 0.4   # keyword gets 40%
    k: int = 6

    def _get_relevant_documents(self, query):
        chroma_docs = self.chroma_retriever.invoke(query)
```

```

bm25_docs = self.bm25_retriever.invoke(query)

seen = set()
scored = []
# add every unique doc from semantic search (weight 0.6)
for doc in chroma_docs:
    ...
    scored.append((content, doc.metadata, self.chroma_weight))
# add keyword results; if already present, BOOST its score
for doc in bm25_docs:
    if content not in seen:
        scored.append((content, doc.metadata, self.bm25_weight))
    else:
        # found by BOTH engines -> score += 0.4 again
        ...
scored.sort(key=lambda x: x[2], reverse=True)
return ... # top k

```

The clever part is the **boost**: a chunk found by *both* engines (content identical) gets extra points — it scored semantically *and* by exact keywords, so it is very likely the right answer and should rise to the top.

Plain English analogy: two detectives search for a witness. One thinks about the *meaning* of your description (semantic), the other looks for people wearing the *exact* clothes you mentioned (keyword). A suspect identified by **both** detectives is far more promising than one found by only one.

Hobbit example: question *"Why did the Elvenking put the dwarves in barrels?"* A chunk from chapter 9, *Barrels Out of Bond*, might be found by BOTH engines (semantic: it's about dwarves in barrels; keyword: it literally contains "barrels", "elves", "dwarves"). Its combined score becomes `0.6 + 0.4 = 1.0`, beating every chunk found by only one engine.

2.8 Step 7 — Polishing the results with Cohere reranking

After the hybrid step, we have 6 candidate chunks. But *retrieved* does not mean *relevant*. The last filter is a **reranker**.

What is reranking? Chroma and BM25 are fast, "cheap" searches that return many candidates but rank them roughly. A reranker is a slower but far more accurate model: it takes the question plus each candidate and gives a careful relevance score from 0 to 1 for *that specific pair*.

The project does this with the Cohere cloud API (only when you set a `COHERE_API_KEY`; otherwise it warns and skips):

```

response = self.client.rerank(
    query=query,
    documents=docs_text,          # the 6 candidates as plain text
    model="rerank-multilingual-v3.0",
    top_n=min(self.top_n, len(documents)),
)

```

Each reordered result gets its score attached to metadata:

```

Document(page_content=doc.page_content,
    metadata={**doc.metadata, "relevance_score": result.relevance_score})

```

A wrapper retriever then runs hybrid search, reranks, and **keeps only passages scoring at least 0.3**, dropping everything else:

```
relevant = [d for d in reranked
             if d.metadata.get("relevance_score", 0) >= self.relevance_threshold]
```

Where and when is reranking used? It is the final quality gate before the text goes to the AI. It ensures the AI only ever sees passages that are genuinely about the question — this is a major defense against the AI writing confident nonsense.

Hobbit example: ask *"Where did Thorin die?"* The hybrid engine may return a passage mentioning Thorin only in passing (e.g., the Battle of Five Armies intro). Cohere scores that chunk low (say 0.12) because it doesn't actually answer WHERE Thorin died, and the 0.3 threshold quietly removes it, while the passage describing *"...Thorin was wounded,... and he died..."* scores high and stays.

3. How the Final Answer Is Generated

Everything so far has just been *searching*. This section shows how the chosen passages turn into the paragraph you read on screen. This is the part that runs for **every** question.

3.1 Step 1 — Ask, and retrieve

`main()` reads your input and hands the question to the retriever:

```
while True:
    question = input("\n> ")
    if question.strip().lower() == "q":
        print("Mellon nath, farewell!") # the farewell message
        break
    if not question.strip():
        continue # skip empty lines
    docs = retriever.invoke(question)
```

`retriever.invoke(question)` runs the *entire* retrieval pipeline from section 2 (hybrid search → rerank → threshold). The result is a list of `Document` objects — the winners of the retrieval stage.

3.2 Step 2 — Format the passages into a context block (`format_passages`)

The AI cannot read Python objects. So each winning document is turned into readable text with a **header** showing its source, using the metadata saved way back in step 2.2:

```
def format_passages(passages):
    parts = []
    for p in passages:
        book = p.metadata.get("book", "Unknown")
        chapter = p.metadata.get("chapter", p.metadata.get("section", ""))
        header = f"[{book}-{chapter}]" if chapter else f"[{book}]"
        parts.append(f"{header}\n{p.page_content}")
    return "\n\n---\n\n".join(parts)
```

Each passage is separated by `---` so the AI can clearly tell one source from the next. This is what the context looks like:

```
[The Hobbit-Chapter 8: Flies and Spiders]
Bilbo drew his little sword... he called it Sting, for it was a sting that he...

---

[The Hobbit-Chapter 12: Inside Information]
...there is a secret entrance to the Lonely Mountain, said he...
```

3.3 Step 3 — The system prompt: the AI's job description

The chatbot's instructions are written in a template. This is where the "personality" and the safety rules live:

```
template = """
You are a world-renowned Tolkien scholar and lore-master...

IMPORTANT:
- If {passages} is empty or contains no relevant information, this question is
  NOT about Tolkien lore. Respond naturally... Do NOT make up an answer.
- If {passages} contains relevant information, answer ONLY using those passages.
- If the passages do not contain enough information to fully answer, say so honestly.
- Never fabricate details that are not present in the provided context.

When answering lore questions:
- Be warm and hospitable...
- Cite the book and chapter when relevant
- Provide direct quotes from the text when possible
...

Question: {question}
"""
```

Notice how carefully the prompt guards against **hallucination**:

- **Empty context** → **not a Tolkien question**. If you ask *"What is the capital of France?"*, the retriever finds nothing relevant, `format_passages` produces almost nothing, and the AI is told to say so instead of inventing Middle-earth facts. (The few non-Tolkien phrases that happen to match will still be passed along — that's why the prompt says to answer **ONLY** from the passages.)
- **Have context** → **answer ONLY from it**. The AI may combine, quote, and explain those passages but never reach beyond them.
- **Not enough info** → **say so**. It is better to admit uncertainty than to guess.

Hobbit example: the template asks the AI to "cite the book and chapter" — so for *"Describe the Arkenstone"*, the answer naturally ends with a reference to *Chapter 16: A Thief in the Night*, because the retrieved passages come from there and the header tells the AI so.

3.4 Step 4 — The AI writes the answer (Qwen2.5 via Ollama)

The composable chain is built with LangChain's `|` operator:

```
model = OllamaLLM(model="qwen2.5:7b-instruct-q4_K_M", client_kwargs={"trust_env": False})
prompt = ChatPromptTemplate.from_template(template)
chain = prompt | model
```

- `prompt | model` means: "fill in the template, then hand it to the model." LangChain calls this LCEL (LangChain Expression Language).
- The model is **Qwen2.5 7B** (a large, capable local model), the *instruct* variant tuned to follow instructions, and the `q4_K_M` quantization version (a compressed copy that runs on normal computers at a small accuracy cost).
- `trust_env: False` tells Ollama to ignore proxy server / environment settings, which can otherwise break local connections.

Then, for each question, the chain is invoked with the two placeholders filled in:

```
results = chain.invoke({
    "passages": format_passages(docs),    # the context block
    "question": question,                 # the user's question
})
print(f"\n{results}")
```

That's it — the model reads the instructions + context + question, and produces the final answer, which is printed.

Hobbit example — the whole loop for one question:

1. You type: `Who did Bilbo name his sword Sting, and why?`
2. Retrieval returns several chunks, topped by *Chapter 8: Flies and Spiders*.
3. `format_passages` wraps them with `[The Hobbit-Chapter 8: Flies and Spiders]` headers.
4. The prompt + context + question go to Qwen2.5.
5. The answer appears, something like:

Bilbo named his sword **Sting** during his fight with the giant spiders in Mirkwood (Chapter 8: Flies and Spiders). As the text says, "he called it Sting, and Sting it was" — a fitting name because it stung the spiders and glowed bluish when great enemies of the elves were near.

3.5 First run vs. every later run

The program supports two situations:

First run (no `chroma_db/` yet):

1. `load_documents()` reads every PDF, smart-splits into chapters, chunks to ~800 characters.
2. `get_vector_store()` embeds all chunks with `mxbai-embed-large` and stores them in `chroma_db/`.
3. A BM25 keyword index is built from the very same in-memory chunks.
4. From then on, it behaves like a "later run".

Later runs (`chroma_db/` already exists):

1. `load_documents()` spots the folder and returns immediately — no re-reading, no re-embedding:

```
add_docs = not os.path.exists(db_location)
if not add_docs:
    return [], False
```

110-138	<code>split_by_chapters</code>	Splits text before each <code>Chapter N</code> heading
141-159	<code>split_by_sections</code>	Splits text on <code>===</code> markers
162-172	<code>smart_split</code>	Chooses the right splitter for each document
175-207	<code>load_documents</code>	Reads PDFs, splits, chunks — only if the DB doesn't exist
210-246	<code>get_vector_store</code>	Embeds chunks and persists them in Chroma
249-256	<code>get_bm25_retriever</code>	Builds the BM25 keyword index
259-307	<code>get_retriever</code>	The master function wiring everything together

4.3 `requirements.txt`

Package	Role in this project
<code>langchain-core</code>	Core building blocks: <code>Document</code> , <code>BaseRetriever</code> , prompts
<code>langchain-community</code>	Community integrations (note: being sunset, used only for utilities)
<code>langchain-ollama</code>	Ollama integration for both the LLM and embeddings
<code>langchain-chroma</code>	The Chroma vector store integration
<code>langchain-text-splitters</code>	<code>RecursiveCharacterTextSplitter</code>
<code>pypdf</code>	Legacy PDF reader (kept for compatibility, not directly used)
<code>PyMuPDF</code>	The real PDF text extractor used by <code>load_documents</code>
<code>fastapi</code> + <code>uvicorn</code>	Web server stack — installed ahead of time, not yet used by the CLI
<code>rank_bm25</code>	The BM25 keyword scoring algorithm
<code>cohere</code>	The Cohere reranking API client
<code>nltk</code>	Word tokenization support for BM25

5. Ways to Make the Whole Project Better

The project works, but there is plenty of room to grow. Ideas are ordered roughly by impact. Each has a short *why* and a practical *how*.

5.1 Add a web interface (FastAPI is already installed)

- **Why:** right now this is a terminal-only chatbot. A web page makes it usable by anyone, and lets answers be displayed with formatting, citations, and clickable sources.
- **How:** `uvicorn` and `fastapi` are already in `requirements.txt`. Create a small `app.py` with a `/chat` endpoint that calls the same `get_retriever()` + `chain` used by `main.py`. The retrieval and

generation logic can be reused almost unchanged.

5.2 Add conversation memory

- **Why:** currently every question is answered in isolation. Follow-ups like *"And what happened to him next?"* make no sense without history.
- **How:** keep recent question/answer pairs and include them in the prompt. LangChain offers `ChatMessageHistory` and memory wrappers; Qwen2.5 is an *instruct* model that handles multi-turn context well. Remember to keep history short so the context limit isn't exceeded.

5.3 Stream the answer as it's generated

- **Why:** waiting 5-10 seconds for a full answer feels slow. Streaming shows words as they appear, which feels dramatically faster.
- **How:** replace `model.invoke(...)` with `model.stream(...)` and print token-by-token. This is an especially nice improvement once a web UI exists (using SSE / websockets).

5.4 Support more books

- **Why:** the prompt already says expertise covers "The Hobbit in full detail." Adding *The Lord of the Rings* and *The Silmarillion* instantly expands the bot's world.
- **How:** add entries to `BOOK_NAME_MAP`, drop the PDFs in `data/`, and delete `chroma_db/` once so the database rebuilds with the new books. Update the persona line in the template to mention the new works.

5.5 Better citations and provenance

- **Why:** telling the user *which PDF page* a fact came from builds trust and helps fact-checking.
- **How:** when extracting text with `pymupdf`, record the page number into each chunk's `metadata["page"]`. Pass the metadata through `format_passages` so citations read like *"[The Hobbit, Ch. 5, p. 84]"*.

5.6 Richer chunking

- **Why:** 800-character chunks are simple but arbitrary. A fact split across two chunks may confuse the retrievers, and long prose fragments lose context.
- **How:** consider *semantic* chunking (split where meaning changes — e.g., using embeddings to detect topic shifts), or chunk per *paragraph* with chapter metadata. Tune `chunk_size/` `chunk_overlap` and evaluate before/after.

5.7 Make the knobs configurable

- **Why:** `chroma_weight=0.6`, `bm25_weight=0.4`, `k=6`, and the 0.3 relevance threshold are hard-coded. Tuning them is currently a code edit.
- **How:** read them from a `.env` file (add `python-dotenv`) or environment variables, e.g. `RAG_K=6`, `RAG_RELEVANCE_THRESHOLD=0.3`, `RAG_CHROMA_WEIGHT=0.6`.

5.8 Build an evaluation harness

- **Why:** you cannot improve what you cannot measure. Right now there is no way to know if a change makes answers better or worse.
- **How:** create a "golden set" of ~30-50 Q&A pairs with known answers from The Hobbit. Write a scoring script that checks retrieval hit-rate and answer quality (manually or with a metric like RAGAS). Run it after every change.

5.9 Add metadata filters

- **Why:** a question like *"What did Gollum say about his precious?"* only needs Chapter 5. Searching the whole library first wastes effort and risks pulling wrong-chapter text.
- **How:** before hybrid search, pre-filter the candidate pool by metadata (e.g., `book == "The Hobbit"` and `chapter` contains "Riddles"). Chroma supports `where` filters on metadata.

5.10 Cache repeating work

- **Why:** embedding the *same* question every time is wasted computation; the same question asked twice should not re-embed or re-rerank.
- **How:** keep a simple question → answer cache (SQLite or in-memory dict), or cache the query embedding so repeated questions skip the embedding step.

5.11 Improve the BM25 side

- **Why:** `doc.page_content.lower().split()` is a very naive tokenizer — "Bilbo's" and "Bilbo," won't match "Bilbo", and there's no stemming.
- **How:** use NLTK's `word_tokenize` (already downloaded) plus a stemmer (Porter or Snowball), and normalize punctuation. This makes keyword matching stronger for names in possessive/plural forms.

5.12 Add a rebuild/versioning command

- **Why:** the database is rebuilt only by deleting `chroma_db/`. If you change the chunk size or add a book, it's easy to forget.
- **How:** store a "build hash" (chunk config + file list + model names) inside Chroma's metadata. On startup, if the hash differs from the current config, rebuild automatically or with `--rebuild` flag.

5.13 Fallback handling for reranking

- **Why:** without `COHERE_API_KEY`, the project silently skips reranking (it prints a warning). The threshold protection disappears.
- **How:** consider a local huggingface cross-encoder as a fallback reranker, or lower reliance on the API by combining the hybrid score with the rerank score instead of hard-thresholding.

5.14 Add unit and end-to-end tests

- **Why:** no tests means small changes can silently break ingestion or retrieval.
- **How:** add `pytest` tests for `smart_split` (does The Hobbit split into chapters?), `extract_book_name`, `HybridRetriever` merging/boost logic, and a mocked `main` loop. This is cheap insurance as the project grows.

5.15 Stronger anti-hallucination measures

- **Why:** the prompt tells the model not to fabricate, but prompts are soft guarantees.
- **How:** (a) require direct quotes by asking for verbatim text plus a citation; (b) post-check that returned facts contain substrings from the passages; (c) lower the rerank threshold when passages are uncertain and ask the model to self-evaluate confidence.

6. Summary Cheat Sheet

Question	Answer
What reads the PDFs?	<code>pymupdf</code> (page-by-page text extraction)
What splits chapters?	<code>smart_split</code> → <code>split_by_chapters</code> (regex lookahead on <code>Chapter N</code>)
What cuts text into buildable pieces?	<code>RecursiveCharacterTextSplitter</code> (800 chars, 200 overlap)
What stores meaning?	Chroma vector DB in <code>chroma_db/</code> , embedded with <code>mxbai-embed-large</code>
What does keyword matching?	BM25 (<code>rank_bm25</code>) via <code>BM25Retriever</code>
What combines both searches?	<code>HybridRetriever</code> (0.6 semantic + 0.4 keyword, boost if in both)
What re-orders / filters finally?	<code>CohereReranker</code> (API) keeping scores ≥ 0.3
What writes the answer?	Qwen2.5 7B (<code>q4_K_M</code>) via Ollama, steered by the system prompt
Why doesn't it hallucinate?	The prompt forbids fabrication, the context gate + threshold filter only pass relevant text
Where does the data live?	<code>data/*.pdf</code> as source; <code>chroma_db/</code> as the built library